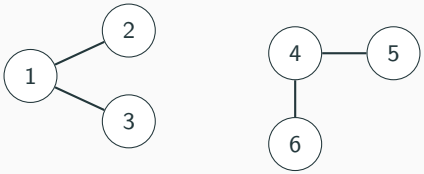
Union-Find

Disjoint Sets with Union by Size and Path Compression

Motivation: Connectivity as Edges Arrive

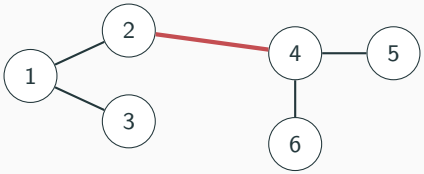
Edges of an undirected graph arrive one at a time. At any moment we want to answer: are u and v in the same connected component?



- Is 3 connected to 5? No.
- Rerunning DFS after each edge costs $O(n + m)$ per query. Too slow.

Motivation: Connectivity as Edges Arrive

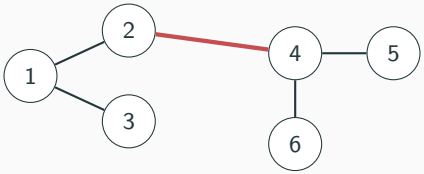
Edges of an undirected graph arrive one at a time. At any moment we want to answer: are u and v in the same connected component?



- Is 3 connected to 5? After adding the edge $\{2, 4\}$: yes.
- Rerunning DFS after each edge costs $O(n + m)$ per query. Too slow.

Motivation: Connectivity as Edges Arrive

Edges of an undirected graph arrive one at a time. At any moment we want to answer: are u and v in the same connected component?



- Is 3 connected to 5? After adding the edge $\{2, 4\}$: yes.
- Rerunning DFS after each edge costs $O(n + m)$ per query. Too slow.

Used in Kruskal's minimum spanning tree algorithm, image segmentation, network connectivity, equivalence of variables in compilers, ...

The Disjoint-Set Interface

Maintain a partition of $\{1, \ldots, n\}$ into disjoint sets. Each set has one representative.

- `find(x)`: return the representative of the set containing x.
- `union(x, y)`: merge the sets containing x and y.

x and y are in the same set $\iff$ `find(x) == find(y)`.



Shaded vertices are the representatives.

First Attempt: Store the Label

Keep an array `label[x]` = representative of x.

x	1	2	3	4	5	6
`label[`x`]`	1	1	1	4	4	6

- `find(x)`: return `label[x]`. $O(1)$
- `union(x, y)`: relabel every element whose label is `label[y]`. $O(n)$

First Attempt: Store the Label

Keep an array `label[x]` = representative of x.

x	1	2	3	4	5	6
`label[`x`]`	1	1	1	4	4	6

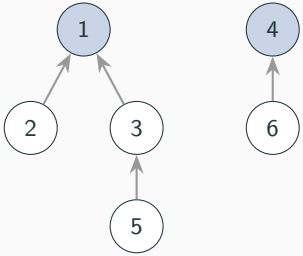
- `find(x)`: return `label[x]`. $O(1)$
- `union(x, y)`: relabel every element whose label is `label[y]`. $O(n)$

$n-1$ unions cost $\Theta(n^2)$ in total. Can we make union cheap without making find expensive?

A Forest of Parent Pointers

Sets as Trees

Store each set as a rooted tree. Every element points to its parent. The root is the representative and points to itself.



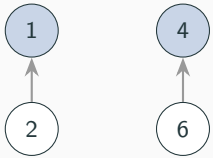
x	1	2	3	4	5	6
`parent[`x`]`	1	1	1	4	3	4

- `find(x)`: follow parent pointers up to the root. Cost = depth of x.
- `union(x, y)`: find both roots and make one point to the other. Cost = two finds $+O(1)$.

Find and Union

```
function find(parent, x)
    while parent[x] != x
        x = parent[x]
    end
    return x
end

function union!(parent, x, y)
    rx = find(parent, x)
    ry = find(parent, y)
    if rx != ry
        parent[ry] = rx
    end
end
```

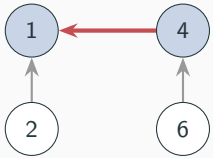


Initially `parent[x] = x` for every x: n singleton trees.

Find and Union

```
function find(parent, x)
    while parent[x] != x
        x = parent[x]
    end
    return x
end

function union!(parent, x, y)
    rx = find(parent, x)
    ry = find(parent, y)
    if rx != ry
        parent[ry] = rx
    end
end
```



`union!(parent, 2, 6)`: roots 1 and 4, so set `parent[4] = 1`.

Initially `parent[x] = x` for every x: n singleton trees.

The Problem: Tall Trees

`union!(parent, 2, 1)`, `union!(parent, 3, 1)`, ..., `union!(parent, n, 1)`:



- Each union hangs the old tree below the new singleton.
- The tree becomes a path of height $n - 1$.
- `find(1)` now costs $\Theta(n)$, and n such finds cost $\Theta(n^2)$.

Two fixes:

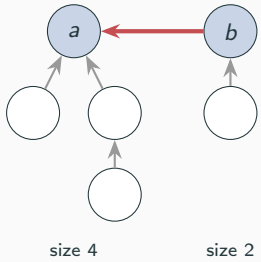
1. Union by size: keep trees shallow when merging.
2. Path compression: flatten trees when searching.

Union by Size

Union by Size

Store `size[r]` = number of elements in the tree with root r. Always attach the smaller tree below the root of the larger one.

```
function union!(parent, size, x, y)
    rx = find(parent, x)
    ry = find(parent, y)
    rx == ry && return
    if size[rx] < size[ry]
        rx, ry = ry, rx # rx is larger
    end
    parent[ry] = rx
    size[rx] += size[ry]
end
```



The size-2 tree goes below a. Ties: either way.

Union by Size Keeps Trees Shallow

Lemma

With union by size, every tree of size s has height at most $\log_2 s$. So every find costs $O(\log n)$.

Union by Size Keeps Trees Shallow

Lemma

With union by size, every tree of size s has height at most $\log_2 s$. So every find costs $O(\log n)$.

Proof. Follow one element x.

- The depth of x grows by 1 only when the root of its tree T is attached below another root, whose tree T' has $|T'| \geq |T|$.
- The merged tree has size $|T| + |T'| \geq 2|T|$: the size of the tree containing x at least doubles.
- The size starts at 1 and never exceeds s, so this happens at most $\log_2 s$ times.

So every element has depth at most $\log_2 s$. □

Union by Size Keeps Trees Shallow

Lemma

With union by size, every tree of size s has height at most $\log_2 s$. So every find costs $O(\log n)$.

Proof. Follow one element x.

- The depth of x grows by 1 only when the root of its tree T is attached below another root, whose tree T' has $|T'| \geq |T|$.
- The merged tree has size $|T| + |T'| \geq 2|T|$: the size of the tree containing x at least doubles.
- The size starts at 1 and never exceeds s, so this happens at most $\log_2 s$ times.

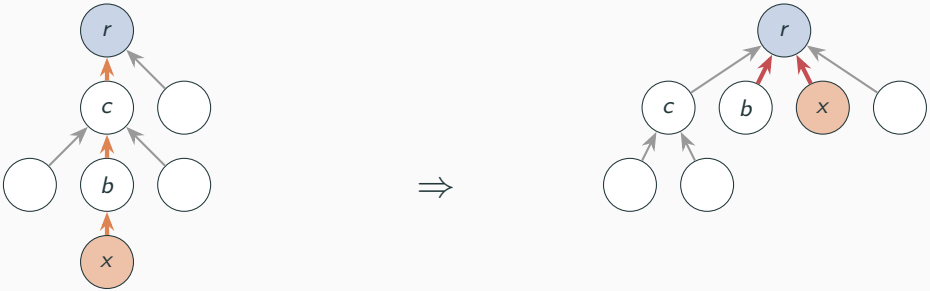
So every element has depth at most $\log_2 s$. □

Union by rank (an upper bound on height instead of size) gives the same guarantee.

Path Compression

Path Compression

During `find(x)` we visit every vertex on the path from x to the root. Make all of them point directly to the root.



before `find(x)` after `find(x)`

The find costs the same this time, but every later find from x, b, or their descendants is faster.

Find with Path Compression

```
function find(parent, x)
    if parent[x] != x
        parent[x] = find(parent, parent[x])
    end
    return parent[x]
end
```

- The recursion walks up to the root.
- On the way back, each vertex on the path gets the root as its new parent.
- One extra assignment per visited vertex, so the cost is still $O(\text{depth})$.

Path compression never changes the root of any set, so `size` only needs to be correct at the roots.

Both Together

Theorem (Tarjan, 1975)

With union by size (or rank) and path compression, any sequence of m operations on n elements takes $O(m\,\alpha(n))$ time.

- $\alpha(n)$ is the inverse Ackermann function. It grows absurdly slowly: $\alpha(n) \leq 4$ for every n smaller than the number of atoms in the universe.
- So in practice each operation is amortized constant time.
- The proof is beyond this course; the $O(\log n)$ bound from union by size alone is the one to remember.

Example

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

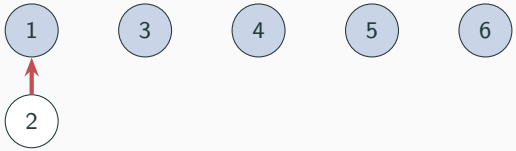


x	1	2	3	4	5	6
`parent[`x`]`	1	2	3	4	5	6

Start: six singletons.

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

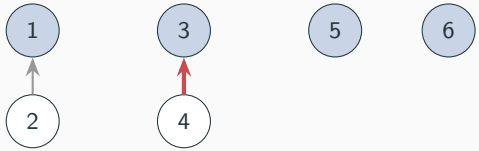


x	1	2	3	4	5	6
`parent[`x`]`	1	**1**	3	4	5	6

`union(1, 2)`: sizes tie, so 2 goes below 1.

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

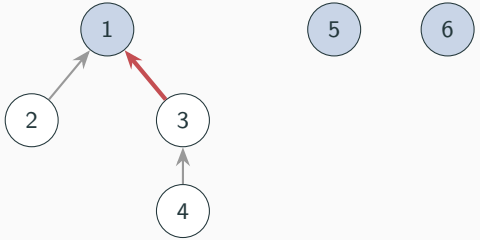


x	1	2	3	4	5	6
`parent[`x`]`	1	1	3	**3**	5	6

`union(3, 4)`: 4 goes below 3.

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

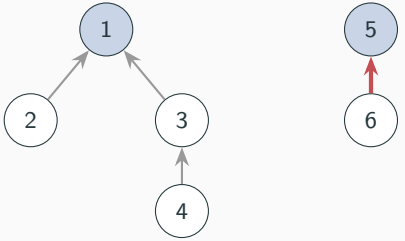


x	1	2	3	4	5	6
`parent[`x`]`	1	1	**1**	3	5	6

`union(2, 4)`: roots 1 and 3, both size 2, so 3 goes below 1.

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

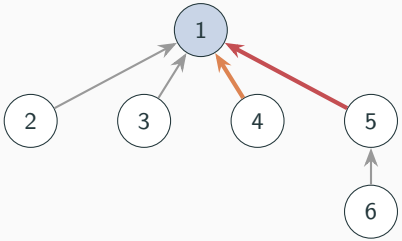


x	1	2	3	4	5	6
`parent[`x`]`	1	1	1	3	5	**5**

`union(5, 6)`: 6 goes below 5.

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.

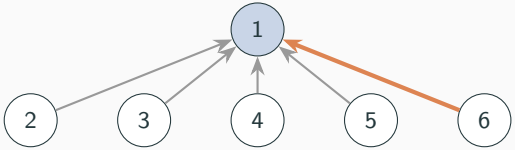


x	1	2	3	4	5	6
`parent[`x`]`	1	1	1	**1**	**1**	5

`union(6, 4)`: `find(4)` compresses $4 \to 3 \to 1$ (orange). Root 5 (size 2) goes below root 1 (size 4).

Example Trace

Union by size (ties: the root of y goes below the root of x), path compression.



x	1	2	3	4	5	6
`parent[`x`]`	1	1	1	1	1	1

`find(6)`: compresses $6 \to 5 \to 1$. Every element now points straight at the root.

Union-Find in Julia

```
struct UnionFind
    parent::Vector{Int}
    size::Vector{Int}
end
UnionFind(n) = UnionFind(collect(1:n), ones(Int, n))
function find(uf::UnionFind, x)
    if uf.parent[x] != x
        uf.parent[x] = find(uf, uf.parent[x])
    end
    return uf.parent[x]
end
function union!(uf::UnionFind, x, y)
    rx, ry = find(uf, x), find(uf, y)
    rx == ry && return false
    if uf.size[rx] < uf.size[ry]; rx, ry = ry, rx; end
    uf.parent[ry] = rx
    uf.size[rx] += uf.size[ry]
    return true
end
```

- The struct is immutable but its vectors are not, so `union!` can modify them.
- `union!` returns whether two different sets were merged. Kruskal's algorithm uses exactly this to decide whether to keep an edge.

Application: Counting Components

Count the connected components of an undirected graph with n vertices and edge list edges.

```
function count_components(n, edges)
    uf = UnionFind(n)
    count = n
    for (u, v) in edges
        if union!(uf, u, v)
            count -= 1
        end
    end
    return count
end
```

- Start with n components.
- Each edge joining two different sets merges two components.
- Time $O(n + m\,\alpha(n))$, and edges can be processed as they arrive.

Exercise

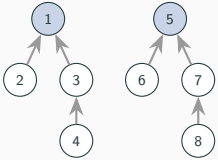
Start with eight singletons $\{1\}, \ldots, \{8\}$. Use union by size (ties: the root of y goes below the root of x) and path compression.

`union(1,2)`, `union(3,4)`, `union(5,6)`, `union(7,8)`,
`union(1,3)`, `union(5,7)`, `union(4,8)`, `find(6)`

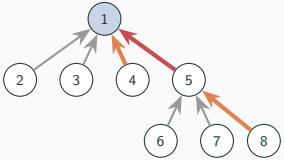
1. Draw the forest after the first six unions.
2. Draw the forest after `union(4,8)`, and again after `find(6)`. Give the final `parent` array.
3. Without path compression, what is the largest possible height of a tree on 8 elements built by union by size?

Solution

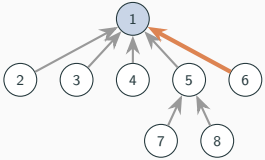
(a) after six unions



(b) after `union(4,8)`



(b) after `find(6)`



In `union(4,8)`, `find(4)` compresses 4 onto 1 and `find(8)` compresses 8 onto 5; the sizes tie at 4, so 5 goes below 1. Final `parent` $= [1, 1, 1, 1, 1, 1, 5, 5]$.

(c) $\log_2 8 = 3$, by the lemma. It is reached by merging equal-sized trees without compression: pairs, then pairs of pairs, then the two halves.

Summary

implementation	`find`	`union`
label array	$O(1)$	$O(n)$
parent forest, naive	$O(n)$	$O(n)$
union by size	$O(\log n)$	$O(\log n)$
union by size + path compression	$O(\alpha(n))$ amortized	

- Each set is a tree of parent pointers; its root is the representative.
- Union by size: attach the smaller tree, so heights stay $\leq \log_2 n$.
- Path compression: point everything on a find path at the root.